

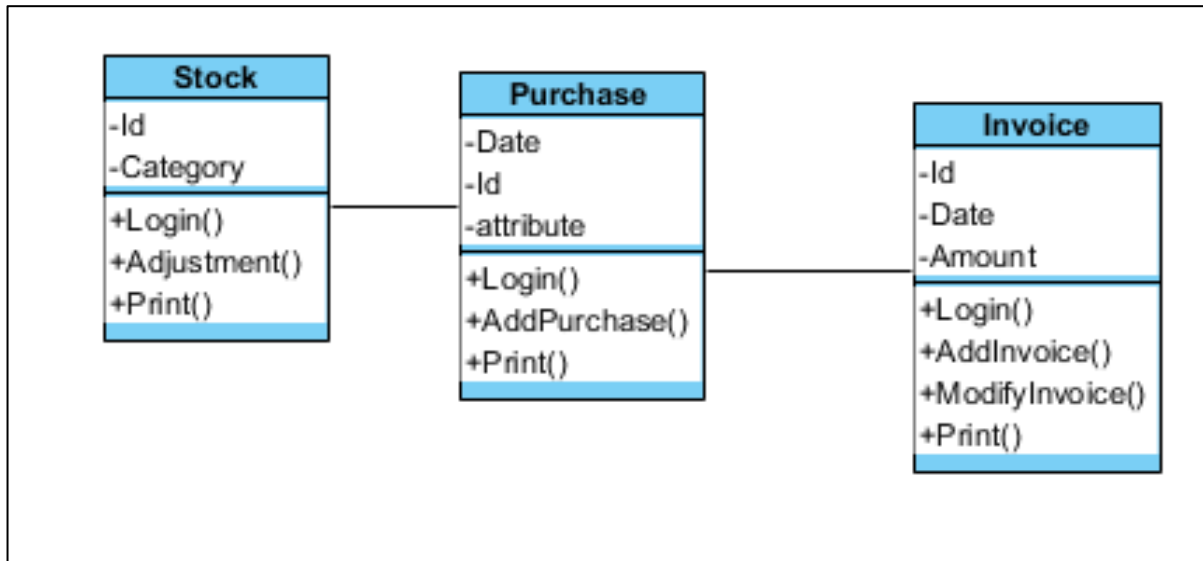
Tutorial 3 – Software Design Strategies and Methods

1. Differentiate between *function-oriented (structured) design* and *object-oriented design*.
2. Compare and contrast between the following terms *top-down* and *bottom-up* in the context of software design.
3. What is the essential difference between components and objects? Why do we need them both?
4. What is *component-based design (CBD)*? Discuss the advantages of *component-based design* over *object-oriented design*.
5. Discuss quality concerns in software design.
6. Identify and describe the different types of stakeholder concern that may arise in a large system? Discuss how can aspects support the implementation of each of these types of concern?
7. Discuss what is meant by *cross-cutting concern*.
8. The implementation of the core concerns in conventional programming languages usually includes additional code to implement the cross-cutting, functional, quality of service, and policy concerns. This leads to two undesirable phenomena: **tangling** and **scattering**. Summarize what is meant by tangling and scattering. Using examples, explain why tangling and scattering can cause problems when system requirements change.
9. Compare and contrast *component-based design* and *service-oriented architecture*.
10.
 - d) A software architect has separated the system into different sub-systems. Each sub-system is made of independent executable files that perform specific group of functions. Each file may embed some API (Application User Interface), sets of libraries and et cetera.
 - (i) Explain the benefits of the strategy applied.

(5 marks)
 - (ii) Describe **THREE (3)** characteristics of the files mentioned that are related to the benefits explain in (d)(i).

(9 marks)

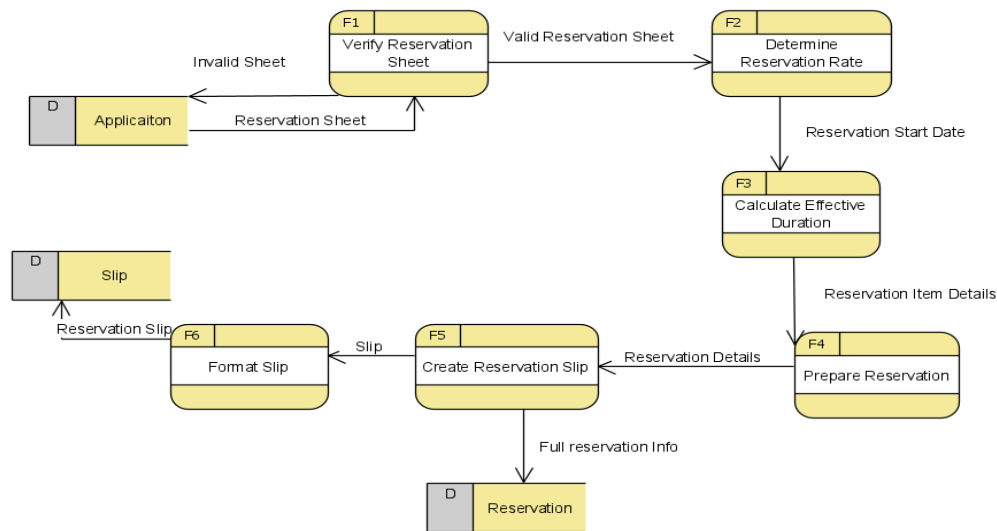
11. The following class diagram shows some classes of a system.



- i) From the class diagram, analyse **TWO (2)** problems in the design.
- ii) Explain how these problems can be solved?

Scenario based exercise (Homework)

a) A library system part is proposed to provide additional online library items reservation features to the student of TARUC. The developer is using functional oriented design and has drawn the DFD (Data Flow Diagram) for this part and the figure below shows the DFD.

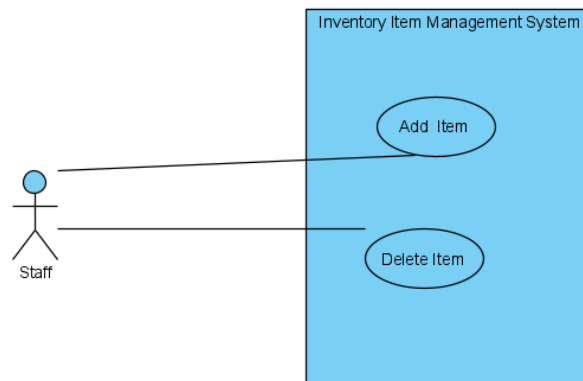


The developer needs to derive structure chart from the DFD, this process called Transform Analysis and basically consists of the following steps:

- 1) Draw DFD
- 2) Identify the central functions of the **DFD**. (Show the central functions) **Find what is meant by *afferent* and *efferent* stream.
- 3) **Transform the DFD into a first-cut structure chart.**
- 4) Refine the **structure chart**.
- 5) Verify that the final **structure chart** meets the requirements of the original **DFD**.

Draw the structure chart based on the above DFD. Show your work to your tutor (if being called).

b) Refer to the following use case diagram:



Use case: Add Item

The Add Item use case is initiated when the staff clicks the "Add" button. A form appears, with title "New Item", containing item's fields for the staff to enter. The process can be terminated by clicking either "OK" or "Cancel". If the "OK" button is clicked, new item is added to the item file. No changes are made if the "Cancel" button is clicked.

Use case: Delete Item

The Delete Item use case is initiated when the staff select the item's name in the item list and clicks the "Delete" button. The system will prompt out a dialog box to ask for confirmation of deleting the item. If the "OK" button is clicked, the item in the item file is deleted, If the "Cancel" button is clicked, no changes are made either to the item file.

Exercise

- i) Identify the objects (class) from the use case.
- ii) Using CRC card, draw and shows the responsibility and collaborators for each object (class).